SecurityX

AI-Powered Cybersecurity Assistant

Capstone Project Report

Submitted by

Sidharth Satapathy

Roll No: 23053005

Batch: Industry Elective (IS30056)

Github_link

Domain: Cybersecurity

Users: Security analysts, IT administrators, SOC teams, and developers

1. Problem Statement

In today's rapidly evolving digital landscape, cybersecurity threats are becoming increasingly sophisticated, frequent, and damaging. Organizations face a constant barrage of attacks including SQL injection, cross-site scripting, ransomware, phishing campaigns, brute force credential attacks, and distributed denial-of-service (DDoS) incidents. Security analysts and IT administrators are often overwhelmed by the volume and complexity of these threats.

The core challenges faced by cybersecurity teams today include:

- Knowledge Overload: The breadth of cybersecurity knowledge required to handle diverse attack vectors is enormous. Analysts must be fluent in attack detection, mitigation, and incident response across dozens of threat categories simultaneously.
- Slow Threat Detection & Triage: Traditional manual analysis of security incidents is time-consuming. Delayed threat identification leads to prolonged exposure windows, increasing the potential damage of an attack.
- Lack of Contextual Guidance: Generic security tools provide alerts but often lack actionable, context-aware guidance. Analysts may know a threat exists but are unsure of the optimal immediate response steps.
- Hallucination in AI Systems: General-purpose large language models (LLMs) can generate plausible-sounding but factually incorrect cybersecurity advice—a critical risk when decisions can affect organizational security posture.
- Adversarial Prompt Exploitation: AI-based security assistants are vulnerable to prompt injection and jailbreak attempts, which could be weaponized to extract harmful information or disable safeguards.

There is a clear and urgent need for an intelligent, reliable, and domain-grounded cybersecurity assistant that can rapidly interpret security incidents, retrieve authoritative guidance from a curated knowledge base, and deliver faithful, actionable recommendations—without hallucinating or being manipulated.

2. Solution / Features

SecurityX is an AI-powered cybersecurity assistant built using a Retrieval-Augmented Generation (RAG) architecture combined with a LangGraph-based agentic workflow. It provides security analysts, developers, and IT administrators with accurate, context-grounded answers to cybersecurity queries and real-time threat detection capabilities.

2.1 Core Architecture

The system is built around an 8-node LangGraph StateGraph that orchestrates the entire question-answering pipeline. Each node handles a specific responsibility, ensuring clean separation of concerns and reliable state transitions.

- Memory Node (Node 1): Maintains a sliding window of up to 6 recent conversation turns, enabling multi-turn contextual dialogue.
- Router Node (Node 2): Uses deterministic keyword matching to classify each query into one of three routes—'retrieve' for knowledge-base lookups, 'tool' for live threat analysis, or 'skip' for memory-only responses. This avoids unnecessary LLM calls for routing decisions.
- Retrieval Node (Node 3): Converts the user query into a vector embedding using SentenceTransformer (all-MiniLM-L6-v2) and performs a semantic similarity search over ChromaDB, returning the top-3 most relevant knowledge base documents.
- Skip Retrieval Node (Node 4): Bypasses the retrieval process for memory-based or tool-only queries, improving response efficiency.

- Threat Detection Tool Node (Node 5): Analyzes user queries for active attack signatures. Detects Brute Force, DDoS, and Phishing threats with severity classification (Low, Medium, High, Critical) and provides specific remediation steps.
- Answer Node (Node 6): Constructs a structured, grounded answer using the Groq-hosted Llama-3.3-70b-versatile model. The system prompt enforces strict context grounding and detects adversarial inputs, refusing to comply with prompt injection or jailbreak attempts.
- Evaluation Node (Node 7): A faithfulness evaluator that rates whether the generated answer is grounded in the retrieved context (score 0.0–1.0). Answers with scores below 0.7 trigger a retry (up to 2 retries) with escalated grounding instructions.
- Save Node (Node 8): Appends the final answer to the conversation history (truncated to 200 characters per turn) and persists it for subsequent queries.

2.2 Knowledge Base

The system includes a curated knowledge base of 11 cybersecurity topic documents covering:

- SQL Injection — attack mechanisms, detection signatures, and parameterized query defenses
- Cross-Site Scripting (XSS) — stored, reflected, and DOM-based variants with CSP mitigations
- Cross-Site Request Forgery (CSRF) — attack flow and CSRF token prevention strategies
- DDoS Attacks — volumetric, protocol, and application-layer attack types with mitigation
- Phishing Attacks — spear phishing, whaling, vishing, smishing with email authentication
- Malware and Trojans — types, delivery vectors, indicators of infection, and removal
- Ransomware Attacks — lifecycle, notable variants, and incident response procedures
- Zero-Day Vulnerabilities — exploitation lifecycle, notable CVEs, and defense-in-depth
- Firewalls and IDS/IPS — types, detection methods, and configuration best practices
- Incident Response Lifecycle — six-phase NIST-aligned framework with key roles
- Brute Force and Credential Attacks — detection indicators and authentication hardening

2.3 Key Features

Multi-Route Query Processing: Intelligently routes queries through the optimal path—semantic retrieval, active threat detection, or conversation memory—without redundant processing.

Active Threat Detection: Real-time analysis of security incident descriptions with automatic threat classification, severity rating, and immediate action recommendations.

Faithfulness Evaluation: A built-in quality gate evaluates every generated answer for factual grounding. Low-quality responses are automatically retried with stricter instructions.

Adversarial Input Protection: Pattern-matching defense against prompt injection, jailbreak attempts, and role-play manipulation, ensuring the agent stays within its security domain.

Conversational Memory: Sliding-window memory maintains context across a conversation session, enabling follow-up questions without repeating context.

Streamlit Web Interface: A clean, real-time chat UI with sidebar knowledge base explorer, metadata display (route, faithfulness score, threat type), and threat alert banners.

Severity Classification: Threat responses are classified into four severity tiers (Low, Medium, High, Critical) with escalating response recommendations.

3. Screenshots

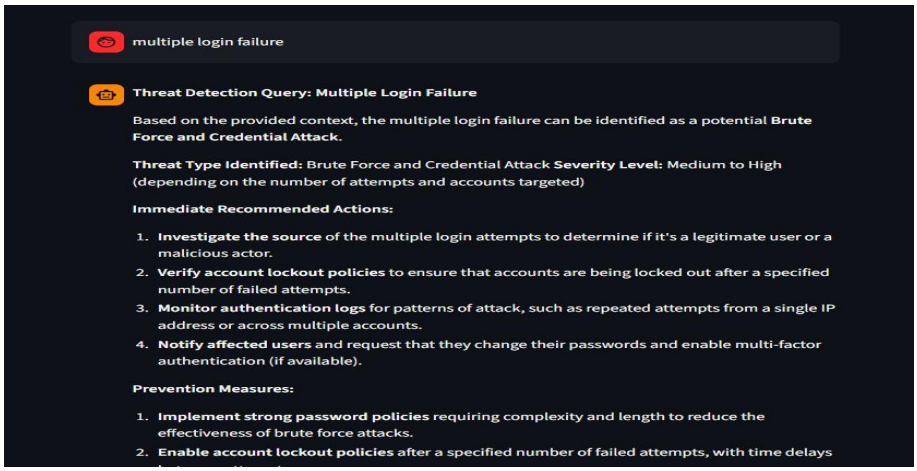
The following screens illustrate the key interfaces and outputs of the SecurityX application.

3.1 Main Chat Interface

The main Streamlit chat interface featuring the conversation panel, sidebar with knowledge base topics, and session controls.

3.3 Active Threat Detection

The threat detection tool in action — analyzing a reported brute force incident and displaying a severity-classified alert with immediate recommended actions.



4. Tech Stack

SecurityX is built entirely in Python using a modern AI/ML stack combining large language models, vector databases, embedding models, and agentic orchestration frameworks. The table below details every component used in the system.

Category	Technology / Tool
Programming Language	Python 3.x
LLM Provider	Groq API (Llama-3.3-70b-versatile)
Agent Framework	LangGraph (StateGraph)
LLM Integration	LangChain (ChatGroq, Messages)
Vector Database	ChromaDB (in-memory)
Embedding Model	SentenceTransformer (all-MiniLM-L6-v2)

Frontend / UI	Streamlit
Environment Config	python-dotenv
Memory Management	LangGraph MemorySaver (checkpointer)

4.1 Component Architecture Overview

The system follows a clean layered architecture:

- Presentation Layer: Streamlit web application (capstone_streamlit.py) handles all user interaction, session state, and display logic.
- Agent Layer: LangGraph StateGraph (agent.py) orchestrates the 8-node pipeline, managing state transitions, routing decisions, and evaluation loops.
- LLM Layer: Groq API hosts the Llama-3.3-70b-versatile model for answer generation and faithfulness evaluation, accessed via LangChain's ChatGroq wrapper.
- Retrieval Layer: ChromaDB (in-memory) stores document embeddings; SentenceTransformer generates query and document embeddings for semantic search.
- Knowledge Layer: 11 domain-expert-curated cybersecurity documents provide the factual grounding for all responses.

5. Unique Points

SecurityX incorporates several design choices that distinguish it from generic chatbots and standard RAG implementations:

5.1 Deterministic Routing Without LLM

Unlike many RAG systems that use a secondary LLM call to decide routing, SecurityX uses deterministic keyword matching for query classification. This eliminates routing latency, reduces token costs, and prevents the router itself from hallucinating routing decisions—a known failure mode in LLM-based routers.

5.2 Self-Evaluating Quality Gate

The faithfulness evaluation loop is built into the graph itself. Every response is scored before it reaches the user. If the score falls below 0.7, the answer node is retried with escalated grounding instructions—up to two times. This closed-loop quality assurance mechanism is rare in production RAG systems and directly addresses the hallucination problem that plagues AI-based security advisors.

5.3 Domain-Specific Threat Detection Tool

Rather than relying solely on the LLM to infer threat context from a query, SecurityX includes a specialized threat_detector_tool that performs pattern analysis of security incident descriptions. This tool produces structured outputs—threat type, severity level, detection basis, and recommended actions—before the LLM even generates a response. The LLM then uses this structured tool output as additional grounded context.

5.4 Adversarial Input Hardening

SecurityX implements a curated list of adversarial prompt patterns that are checked before every LLM invocation. Queries containing prompt injection indicators ('ignore previous instructions', 'act as', 'jailbreak', etc.) are intercepted at the answer node and receive a canned refusal response, preventing the assistant from being manipulated into providing harmful information.

5.5 Sliding Window Memory with Truncation

The conversation history uses a sliding window of the last 6 messages, with answer snippets truncated to 200 characters. This design prevents context window overflow in long conversations while preserving the most recent conversational context for coherent multi-turn dialogue—balancing memory richness against token efficiency.

5.6 Separation of Routing and LLM Concerns

The StateGraph architecture cleanly separates routing logic, retrieval, tool use, generation, and evaluation into distinct nodes. Each node is independently testable and replaceable. This modularity means the knowledge base can be swapped, the LLM can be changed, and the evaluation criteria can be updated without refactoring the entire system.

6. Future Improvements

While SecurityX is a functional and well-architected cybersecurity assistant, several enhancements would significantly expand its capabilities and production readiness:

6.1 Expanded Knowledge Base

The current knowledge base covers 11 core cybersecurity topics. A production-grade system would incorporate hundreds of documents from authoritative sources including NIST SP 800-series publications, OWASP Top 10 detailed guides, CVE database entries, MITRE ATT&CK framework techniques, and real-time threat intelligence feeds from vendors like CrowdStrike and Palo Alto Networks. Integration with live CVE/NVD APIs would enable the assistant to answer questions about newly disclosed vulnerabilities.

6.2 Persistent Vector Store

The current ChromaDB instance is in-memory and resets on each application restart. Migrating to a persistent ChromaDB deployment or a cloud-hosted vector database such as Pinecone, Weaviate, or Qdrant would enable the knowledge base to grow incrementally without re-ingestion on every startup. This would also support document versioning and update pipelines.

6.3 Multi-Modal Input Support

Future versions could accept screenshots of security dashboards, SIEM alert panels, firewall log exports, and network traffic captures as input. Vision-language models could extract threat indicators from these images, enabling SecurityX to analyze visual evidence of attacks—not just text descriptions.

6.4 Role-Based Access and Audit Logging

For enterprise deployments, SecurityX should implement role-based access control (RBAC) to restrict sensitive threat intelligence to authorized personnel. Additionally, comprehensive audit logging of all queries and responses—with timestamps, user identifiers, and faithfulness scores—would support compliance requirements and security operations center (SOC) workflows.

6.5 Threat Intelligence Feedback Loop

Analysts should be able to rate the quality of SecurityX's responses and flag incorrect or incomplete guidance. This feedback could be used to automatically identify knowledge base gaps, fine-tune the faithfulness evaluator thresholds, and prioritize which new topics to add to the knowledge base—creating a continuously improving system.

— End of Report —